

Bit Manipulation in Java

A structured interview-focused Bit Manipulation handbook containing core concepts, important tricks, intuition, dry runs, pattern recognition, and optimized solutions for revision and placements.

Table of Contents

1. Introduction
 2. Why Bit Manipulation Matters
 3. Binary Fundamentals
 4. Bitwise Operators Cheatsheet
 5. Important Bit Tricks
 6. Core Problems & Solutions
 7. XOR Mastery
 8. Pattern Recognition Guide
 9. Common Mistakes
 10. Complexity Cheatsheet
 11. Real World Applications
 12. Rapid Revision Section
-

Introduction

Bit Manipulation is the process of directly operating on the binary representation of numbers using bitwise operators.

Since computers internally store data in binary format, bit manipulation allows extremely fast and optimized computations.

It is widely used in:

- Competitive Programming
 - Technical Interviews
 - Cryptography
 - Compression Algorithms
 - Operating Systems
 - Networking
 - Game Engines
 - Embedded Systems
-

Why Bit Manipulation Matters

Advantages

- Faster than many arithmetic operations
- Reduces time complexity in many problems
- Helps optimize memory and computation
- Frequently asked in coding interviews
- Builds strong low-level problem-solving intuition

Binary Fundamentals

Decimal to Binary

Decimal	Binary
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000

Bit Manipulation Fundamentals: MSB, LSB & Negative Numbers

What is a Bit?

A bit is the smallest unit of data in a computer.

0 → OFF

1 → ON

Example:

13 = 1101₂

MSB (Most Significant Bit)

The leftmost bit in a binary number.

```
13 = 1101
    ↑
    MSB
```

Why is it important?

- Represents the highest power of 2.
- In signed numbers, it acts as the sign bit.

Examples

```
13 = 1101
MSB = 1

5  = 0101
MSB = 0
```

▽ LSB (Least Significant Bit)

The rightmost bit in a binary number.

```
13 = 1101
    ↑
    LSB
```

Why is it important?

Used to determine:

- Even/Odd numbers
- Last binary digit

Quick Check

```
if((n & 1) == 1)
```

Then number is Odd.

```
if((n &1)==0)
```

Then number is Even.

Examples

```
10 = 1010 → LSB = 0 → Even
```

```
11 = 1011 → LSB = 1 → Odd
```

⚡ Signed vs Unsigned Numbers

Unsigned

All bits represent magnitude.

8-bit

```
00000000 = 0
```

```
11111111 = 255
```

Range:

```
0 to 255
```

Signed

One bit is reserved for sign.

```
MSB = 0 → Positive
```

```
MSB = 1 → Negative
```

Range for 8-bit:

-128 to +127

How Negative Numbers Are Represented

Computers use:

Two's Complement

Not Sign-Magnitude Not One's Complement

Two's Complement

Step 1: Write Positive Binary

Example:

5 = 00000101

Step 2: Invert Bits

00000101

↓

11111010

Step 3: Add 1

11111010

+ 1

11111011

Result:

```
-5 = 11111011
```

Verify

Take:

```
11111011
```

Invert:

```
00000100
```

Add 1:

```
00000101
```

Which is:

```
5
```

Therefore:

```
11111011 = -5
```

Why Two's Complement?

Because arithmetic becomes easy.

Example:

```
5 + (-5)
```

```
00000101
```

```
11111011
```

```
-----  
00000000
```

Result:

0

No special subtraction circuitry required.

Sign Bit

For signed integers:

MSB = 0 → Positive

MSB = 1 → Negative

Examples:

00000101 = +5

11111011 = -5

Important Bit Manipulation Concepts

Check Kth Bit

```
(n & (1 << k)) != 0
```

Set Kth Bit

```
n | (1 << k)
```

Clear Kth Bit

```
n & ~(1 << k)
```

Toggle Kth Bit

```
n ^ (1 << k)
```

Remove Last Set Bit

```
n & (n - 1)
```

Example

```
12 = 1100
11 = 1011
1100
1011
----
1000
```

Check Power of 2

```
(n & (n - 1)) == 0
```

Condition:

```
n > 0
```

Count Set Bits (Brian Kernighan)

```
while(n > 0)
{
    n = n & (n - 1);
}
```



```
count++;  
}
```

Interview Facts

1. Why does $n \& (n - 1)$ work?

Because subtracting 1 flips:

- Rightmost set bit $\rightarrow 0$
- All bits after it $\rightarrow 1$

AND operation removes the last set bit.

2. Why is LSB used for Odd/Even?

Because:

```
Even numbers end with 0
```

```
Odd numbers end with 1
```

in binary.

3. Why Two's Complement instead of Sign-Magnitude?

- Single representation of zero
 - Easier arithmetic operations
 - Faster hardware implementation
-

Memory Tricks

MSB

```
M  $\rightarrow$  Most
```

```
Leftmost bit
```

Think:

"Big Boss Bit"

LSB

L → Least

Rightmost bit

Think:

"Last Bit"

Two's Complement

Invert + Add 1

Remember:

Negative Number
=
Flip All Bits
+
1

 Powers of 2 Table

Power	Value
2 ⁰	1
2 ¹	2
2 ²	4
2 ³	8
2 ⁴	16
2 ⁵	32

Power	Value
2^6	64
2^7	128

Binary ↔ Decimal Conversion

Problem

Learn how to convert:

1. Decimal → Binary
2. Binary → Decimal

These conversions form the foundation of Bit Manipulation because computers internally store everything in binary.

Golden Trick

Decimal → Binary

Keep dividing by 2 and store remainders.

```
Remainders (bottom to top) = Binary Number
```

Binary → Decimal

Multiply each bit by its corresponding power of 2.

```
Bit × 2^Position
```

Then add everything.

Decimal to Binary

Example

Convert:

13 → Binary

Repeated Division

$$13 \div 2 = 6 \quad R = 1$$

$$6 \div 2 = 3 \quad R = 0$$

$$3 \div 2 = 1 \quad R = 1$$

$$1 \div 2 = 0 \quad R = 1$$

Read remainders from bottom to top:

1101

Therefore:

$$13 = 1101_2$$



Decimal → Binary Logic

```
while(n >0)
{
    binary.

    append(n %2);

    n /=2;
}

binary.

reverse();
```



Dry Run

Convert:

10

$$10 \div 2 = 5 \quad R=0$$

$$5 \div 2 = 2 \quad R=1$$

$$2 \div 2 = 1 \quad R=0$$

$$1 \div 2 = 0 \quad R=1$$

Bottom to Top:

1010

Answer:

$$10 = 1010_2$$

Binary to Decimal

Example

Convert:

1101₂

Assign powers of 2:

1	1	0	1
↑	↑	↑	↑
2 ³	2 ²	2 ¹	2 ⁰

Calculate:

$$1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$8 + 4 + 0 + 1$

13

Therefore:

 $1101_2 = 13$

Binary → Decimal Logic

```
int decimal = 0;
int power = 0;

while(binary > 0)
{
    int lastDigit = binary % 10;

    decimal += lastDigit * (1 << power);

    power++;

    binary /= 10;
}
```

Dry Run

Convert:

 1010_2 $0 \times 2^0 = 0$ $1 \times 2^1 = 2$ $0 \times 2^2 = 0$

$$1 \times 2^3 = 8$$

Sum:

$$8 + 2 = 10$$

Answer:

$$1010_2 = 10$$

Alternative Binary → Decimal Logic

Instead of:

```
lastDigit *Math.  
pow(2,power)
```

Use:

```
lastDigit *(1<<power)
```

Because:

```
1 << power = 2^power
```

and bit shifting is faster.

Understanding Positions

Binary Number:

```
110101
```

Bit	Power	Value
1	2^5	32
1	2^4	16
0	2^3	0
1	2^2	4
0	2^1	0
1	2^0	1

Total:

32 + 16 + 4 + 1 = 53

X Common Mistakes

Mistake 1

Reading decimal-to-binary remainders from top to bottom.

Wrong:

13 → 1011

Correct:

13 → 1101

Always read from:

Bottom → Top

Mistake 2

Starting powers from 1.

Wrong:

$2^1, 2^2, 2^3 \dots$

Correct:

 $2^0, 2^1, 2^2 \dots$

LSB always starts at:

 2^0

Why Powers of 2?

Decimal:

347

 $3 \times 10^2 + 4 \times 10^1 + 7 \times 10^0$

Binary:

1101

 $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$

Binary is simply a base-2 number system.

Edge Cases

Decimal = 0

 $0 \rightarrow 0$

Binary = 0

$0 \rightarrow 0$

Leading Zeros

 001101

Same as:

 1101

Leading zeros do not affect value.



Complexity

Decimal $\rightarrow$ Binary

 $O(\log_2 N)$

Binary $\rightarrow$ Decimal

 $O(\text{Number of Bits})$ 

Interview Takeaway

Decimal $\rightarrow$ Binary

Keep dividing by 2 and store remainders.

Binary $\rightarrow$ Decimal

Multiply each bit by its corresponding power of 2 and add.

Important Identity

 $1 \ll k$

means

2^k

This identity appears in almost every Bit Manipulation problem.

Memory Trick

Decimal → Binary

Divide by 2
Collect Remainders
Read Bottom to Top

Think:

 "Division → Remainders → Reverse"

Binary → Decimal

Bit × Power of 2

Think:

 "Multiply → Add → Decimal"

Quick Revision Sheet

Conversion	Trick
Decimal → Binary	Divide by 2 repeatedly
Binary → Decimal	Bit × Power of 2
2^k	$1 \ll k$
LSB Position	2^0
MSB Position	Highest Power of 2
Time Complexity	$O(\log N)$

Bitwise Operators Cheatsheet

Operator	Name	Purpose
&	AND	Checking bits
	OR	Setting bits
^	XOR	Toggling / Cancellation
~	NOT	Inverting bits
<<	Left Shift	Multiply by 2
>>	Right Shift	Divide by 2

Integer Overflow and Wrap Around using Bits

Problem

Why does storing a value outside the range of a data type produce unexpected results?

Example:

```
byte x = (byte) 129;  
System.out.  
  
println(x);
```

Output:

```
-127
```

Why does 129 become -127?

Golden Trick

A computer stores numbers using a **fixed number of bits**.

For an 8-bit signed byte:

```
Range = -128 to 127
```

Only **256 unique bit patterns** are possible:

$$2^8 = 256$$

When a value exceeds the maximum range, the extra carry bit is discarded and the value **wraps around**.

Think of it like a clock:

$$\begin{aligned} 11 + 1 &= 12 \\ 12 + 1 &= 1 \end{aligned}$$

Similarly:

$$127 + 1 = -128$$

Code

```
public class Main {  
    public static void main(String[] args) {  
  
        byte a = 127;  
        a++;  
  
        System.out.println(a);  
  
        byte b = (byte) 129;  
        System.out.println(b);  
    }  
}
```

Output:

```
-128  
-127
```

Intuition

Computers do not store numbers directly.

They store only **bits**.

For example:

```
127 = 01111111
```

When we add 1:

```
01111111
+      1
-----
10000000
```

The result becomes:

```
10000000
```

In Two's Complement representation:

```
10000000 = -128
```

Therefore:

```
127 + 1 = -128
```

The number wraps around because there is no ninth bit available.

Dry Run

Example 1: $127 + 1$

Step 1:

```
127 = 01111111
```

Step 2:

```
01111111
+00000001
-----
10000000
```

Step 3:

Interpret result as signed byte:

$$10000000 = -128$$

Final Answer:

$$127 + 1 = -128$$

Example 2: Storing 129

Binary representation:

$$129 = 10000001$$

Stored bits:

10000001

Since the first bit is 1, it is interpreted as a negative number.

Using Two's Complement:

Invert bits:

01111110

Add 1:

01111111

Value:

127

Therefore:

```
10000001 = -127
```

Final Answer:

```
129 becomes -127
```

✖ Counter Example

Unsigned Byte

If the byte were unsigned:

```
10000001
```

would be:

```
128 + 1 = 129
```

Result:

```
129
```

No negative value appears.

The same bits can represent different values depending on how they are interpreted.

💧 Why It Works

Fixed Bit Storage

An 8-bit number can store only:

```
00000000  
to  
11111111
```

Total combinations:


```
256
```

The CPU cannot magically create extra bits.

When overflow occurs:

```
11111111 + 1
```

becomes:

```
1 00000000
```

The extra carry:

```
1
```

is discarded.

Only:

```
00000000
```

remains.

This causes the value to wrap around.

Two's Complement Representation

Signed integers use Two's Complement.

Positive numbers:

```
00000000 = 0
00000001 = 1
...
01111111 = 127
```

Negative numbers:

```
11111111 = -1
11111110 = -2
...
10000000 = -128
```

This allows addition and subtraction to work using the same hardware circuitry.

⚠ Edge Cases

Maximum Positive Value

```
127 + 1 = -128
```

Minimum Negative Value

```
-128 - 1 = 127
```

Binary:

```
10000000
-      1
-----
01111111
```

Result:

```
127
```

Large Positive Numbers

```
(byte)130 = -126
(byte)131 = -125
(byte)132 = -124
```

Large Negative Numbers

```
(byte)-129 = 127
(byte)-130 = 126
```

Complexity

Time Complexity:

```
O(1)
```

Space Complexity:

```
O(1)
```

Since overflow handling is performed directly by hardware.

Interview Takeaway

Interview Question:

```
Why does 127 + 1 become -128 in Java byte?
```

Answer:

```
A byte uses 8 bits and follows Two's Complement representation.
```

```
127 is stored as:
```

```
01111111
```

```
Adding 1 produces:
```

```
10000000
```

```
which represents -128.
```

```
The overflow occurs because the carry bit is discarded due to the fixed 8-bit storage.
```

Memory Trick

Imagine an 8-bit signed integer as a circular wheel:

```
... -3
... -2
... -1
0
1
2
...
126
127
↓
-128
↓
-127
...
```

After reaching the maximum value:

```
127
```

the next value becomes:

```
-128
```

because the bit pattern wraps around.

One-Line Rule

```
Integer Overflow = Fixed Number of Bits + Extra Carry Bit Discarded
```

Understanding Operators

AND Operator &

Returns 1 only when both bits are 1.

```
1 & 1 = 1
1 & 0 = 0
0 & 1 = 0
0 & 0 = 0
```

Example

```
5 = 0101
1 = 0001
-----
    0001
```

Mainly used for:

- checking bits
- odd/even checks
- masking

OR Operator |

Returns 1 if any bit is 1.

Example

```
5 = 0101
1 = 0001
-----
    0101
```

Mainly used for:

- setting bits

XOR ^

XOR (Exclusive OR) compares two bits and returns 1 only when the bits are different.

XOR Truth Table

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Rule

```
Same Bits      → 0
Different Bits → 1
```

Example

```
5 = 0101
3 = 0011

5 ^ 3

0101
0011
----
0110

= 6
```

Why XOR is Special

Unlike AND and OR, XOR has several unique properties that make it extremely useful in DSA, Competitive Programming, and Bit Manipulation.

Property 1: Same Numbers Cancel Out

```
a ^ a = 0
```

Example

```
5 ^ 5

0101
0101
----
0000

= 0
```

Since every bit is identical, all bits become 0.

Property 2: XOR with 0 Returns the Same Number

$$a \wedge 0 = a$$

Example

$$\begin{array}{r} 5 \wedge 0 \\ 0101 \\ 0000 \\ ---- \\ 0101 \\ = 5 \end{array}$$

Zero does not affect the value.

Property 3: Reversible Operation

$$(a \wedge b) \wedge b = a$$

Example

$$\begin{array}{l} 5 \wedge 3 = 6 \\ 6 \wedge 3 = 5 \end{array}$$

The second XOR cancels the first one.

This property is the foundation of XOR-based swapping and many advanced algorithms.

Property 4: Order Doesn't Matter

$$a \wedge b = b \wedge a$$

Example

$$\begin{array}{l} 5 \wedge 3 = 6 \\ 3 \wedge 5 = 6 \end{array}$$

This property is called **Commutativity**.

Property 5: Grouping Doesn't Matter

$$(a \wedge b) \wedge c = a \wedge (b \wedge c)$$

Example

$$\begin{aligned} (2 \wedge 3) \wedge 4 \\ = \\ 2 \wedge (3 \wedge 4) \end{aligned}$$

This property is called **Associativity**.

XOR for Toggling Bits

XOR can flip a bit.

Example

$$\begin{aligned} \text{Bit} &= 1 \\ 1 \wedge 1 &= 0 \end{aligned}$$

$$\begin{aligned} \text{Bit} &= 0 \\ 0 \wedge 1 &= 1 \end{aligned}$$

A bit XORed with 1 always changes its state.

Toggle ith Bit

Formula

$$n \wedge (1 \ll i)$$

Example


```
n = 10

1010

Toggle bit at index 1

Mask = 0010

1010
0010
----
1000

= 8
```

Finding a Unique Element

When every element appears twice except one:

```
2 3 4 3 2
```

Using XOR:

```
2 ^ 3 ^ 4 ^ 3 ^ 2

= (2 ^ 2) ^ (3 ^ 3) ^ 4

= 0 ^ 0 ^ 4

= 4
```

Result

```
Unique Element = 4
```

This is one of the most common interview questions.

Swapping Two Numbers Using XOR

Logic

```
a =a ^b;  
b =a ^b;  
a =a ^b;
```

Example

```
a = 5  
b = 3  
  
After swapping:  
  
a = 3  
b = 5
```

Quick Comparison

Expression	Result
$a \wedge a$	0
$a \wedge 0$	a
$(a \wedge b) \wedge b$	a
$a \wedge b$	Set bits where bits differ
$n \wedge (1 \ll i)$	Toggle ith bit

 Memory Trick

```
XOR = Difference Detector  
  
Same Bits      → 0  
Different Bits → 1  
  
a ^ a = 0  
a ^ 0 = a  
  
XOR Cancels Duplicates
```

One-Line Summary

```
XOR returns 1 for different bits and 0 for same bits, making it perfect for  
toggling bits, finding unique elements, and cancelling duplicate values.
```

NOT Operator ~

Flips all bits.

$$\sim 1010 = 0101$$

Shift Operators

Shift operators move the bits of a number left or right. Since binary positions represent powers of 2, shifting bits can efficiently perform multiplication and division.

Left Shift <<

Moves all bits to the left by a specified number of positions.

Formula

$$x \ll n = x \times 2^n$$

Example

$$5 = 00000101$$

$$5 \ll 1$$

$$00000101$$

↓

$$00001010$$

$$= 10$$

More Examples

$$7 \ll 1 = 14$$

$$7 \ll 2 = 28$$

$$7 \ll 3 = 56$$

Key Points

- Bits move towards the left.

- Empty positions on the right are filled with 0.
- Leftmost bits that go beyond the available space are discarded.
- Each left shift doubles the value.

```
x << 1 = x × 2
x << 2 = x × 4
x << 3 = x × 8
```

Right Shift >>

Moves all bits to the right by a specified number of positions.

Formula

```
x >> n = x ÷ 2n
```

Example

```
8 = 00001000

8 >> 1

00001000
↓
00000100

= 4
```

More Examples

```
20 >> 1 = 10
20 >> 2 = 5
```

Key Points

- Bits move towards the right.
- Rightmost bits are discarded.
- Each right shift halves the value.
- Performs integer division, so decimal values are lost.

```
13 >> 1 = 6

13 / 2 = 6.5
```

The `.5` is discarded because bits cannot store fractional parts in integer representation.

Creating Bit Masks

A very common use of left shift is creating masks.

Formula

```
1<<i
```

Examples

```
1 << 0 = 0001
1 << 1 = 0010
1 << 2 = 0100
1 << 3 = 1000
```

Only the `ith` bit is set to `1`.

Quick Comparison

Operator	Meaning	Formula
<code>x << n</code>	Move bits left	<code>x × 2ⁿ</code>
<code>x >> n</code>	Move bits right	<code>x ÷ 2ⁿ</code>
<code>1 << i</code>	Create mask	Set only ith bit

Memory Trick

```
Left Shift (<<)
← Move Left
↑ Value Increases
× 2n

Right Shift (>>)
→ Move Right
```

↓ Value Decreases
÷ 2ⁿ

Important Bit Tricks

Trick	Purpose
<code>n & 1</code>	Check Odd / Even
<code>1 << i</code>	Create bit mask
<code>n mask</code>	Set ith bit
<code>n & ~mask</code>	Clear ith bit
<code>n ^ mask</code>	Toggle ith bit
<code>n & (n - 1)</code>	Remove last set bit
<code>(n & (n - 1)) == 0</code>	Check Power of 2

Core Problems & Solutions

1 Check Odd or Even

Problem

Determine whether a number is odd or even.

Intuition

The last bit determines parity.

- Odd numbers end with `1`
- Even numbers end with `0`

Trick Used

`(n & 1)`

Logic

AND with 1 isolates the last bit.

Binary Dry Run

```
7 = 111
1 = 001
-----
    001
```

Result = 1 → Odd

Complexity

- Time: $O(1)$
 - Space: $O(1)$
-

Get ith Bit

Problem

Find whether the ith bit is set or not.

Trick Used

```
(n & (1 << i))
```

Intuition

Create a mask with only the ith bit set.

Logic

If AND result is non-zero, ith bit is set.

Example

```
n = 10 = 1010
i = 1
```

```
Mask = 0010
```

```
1010
0010
-----
0010
```

Bit is set.

3 Set Kth Bit

Problem: Set the K-th Bit

Given an integer n and a bit position k , set the K -th bit of the number and return the resulting value.

Golden Trick

Use the bitwise OR operator with a mask containing only the K -th bit set.

```
n | (1 << k)
```

The OR operation guarantees that the K -th bit becomes 1 , regardless of its previous value.

Intuition

To set a specific bit, we first create a mask:

```
1 << k
```

This shifts 1 to the K -th position.

For example:

```
k = 3

0001 << 3 = 1000
```

Now perform OR:


```
n | mask
```

Since:

```
0 | 1 = 1
1 | 1 = 1
```

the K-th bit is guaranteed to become **1**.

Dry Run

Input

```
n = 10
k = 0
```

Binary representation:

```
10 = 1010
```

Step 1: Create Mask

```
1 << 0 = 0001
```

Step 2: Apply OR

```
1010
0001
----
1011
```

Result

```
11
```

Counter Example

Suppose the bit is already set.

Input

```
n = 13  
k = 2
```

Binary:

```
13 = 1101
```

Mask:

```
0100
```

Apply OR:

```
1101  
0100  
----  
1101
```

Result:

```
13
```

The number remains unchanged because the bit was already set.

Why It Works

The mask contains exactly one bit set:

```
0001 << k
```

When OR is applied:

```
n | mask
```

only the K-th position is affected.

For the target bit:

```
0 | 1 = 1
1 | 1 = 1
```

Therefore, the K-th bit always becomes 1.

All other bits remain unchanged.

Edge Cases

1. Bit Already Set

```
n = 13
k = 2
```

Result remains:

```
13
```

2. Number is Zero

```
n = 0
k = 3
```

```
0000 | 1000 = 1000
```

Result:

```
8
```

3. Setting the 0th Bit

```
n = 10  
k = 0
```

```
1010 | 0001 = 1011
```

Result:

```
11
```



Complexity

Operation	Complexity
Time	$O(1)$
Space	$O(1)$



Interview Takeaway

Whenever you need to force a bit to become 1, use:

```
n | (1 << k)
```

This is the standard and most efficient approach used in bit manipulation problems.



Memory Trick

Think of OR as a permanent marker .

If you draw a 1 over a bit:

```
0 → 1  
1 → 1
```

it always stays 1.

So remember:

```
Set Bit
=
OR with Shifted 1
```

```
n | (1<<k)
```

Whenever you hear "**set the K-th bit**", your brain should immediately think:

👉 **OR + Shifted 1**

4 Clear kth Bit

📌 Problem: Clear the K-th Bit

Given an integer `n` and a bit position `k`, clear the K-th bit of the number and return the resulting value.

Clearing a bit means forcing it to become 0 regardless of its current value.

⚡ Golden Trick

Create a mask with only the K-th bit set:

```
1<<k
```

Invert the mask:

```
~(1<<k)
```

Then perform AND with the original number:

```
n & ~(1<<k)
```

This clears the K-th bit while leaving all other bits unchanged.

Intuition

To clear a bit, we need a mask that contains:

```
1 everywhere
0 at the K-th position
```

Example:

```
k = 2

1 << 2 = 0100
```

Invert it:

```
1011
```

Now perform AND:

```
1101
1011
----
1001
```

The K-th bit becomes 0.

Dry Run

Input

```
n = 13
k = 2
```

Binary representation:

```
13 = 1101
```

Step 1: Create Mask

```
1 << 2 = 0100
```

Step 2: Invert Mask

```
~0100 = 1011
```

Step 3: Apply AND

```
1101  
1011  
----  
1001
```

Result

```
9
```

✗ Common Mistake I Made

Initially, I used:

```
n | ~(1 << k)
```

instead of:

```
n & ~(1 << k)
```

Why This Is Wrong

The inverted mask contains many leading **1**s because Java stores integers using ****32-bit Two's Complement representation****.

Example:

```
1 << 2 = 00000000 00000000 00000000 00000100
```

After inversion:

```
~(1 << 2)  
  
11111111 11111111 11111111 11111011
```

Notice that almost every bit becomes 1.

Now if we use OR:

```
n | ~(1 << k)
```

Example:

```
1101  
11111111111111111111111111111111  
-----  
11111111111111111111111111111111
```

Result:

```
-1
```

or another negative number depending on the input.



Why Was I Getting Negative Numbers?

Java stores integers using **Two's Complement**.

In a 32-bit integer:

```
Most Significant Bit (MSB)
```

acts as the **sign bit**.


```
0 → Positive  
1 → Negative
```

When we applied:

```
n | ~(1 << k)
```

the inverted mask already had:

```
11111111 11111111 11111111 xxxxxxxx
```

The sign bit became:

```
1
```

which tells Java:

```
"This number is negative."
```

Therefore the result often became:

```
-1  
-5  
-9  
...
```

instead of the expected positive value.

Why AND Works

The inverted mask contains:

```
1 everywhere  
0 at the K-th bit
```

AND has the property:

```
1 & x = x
0 & x = 0
```

Therefore:

- The K-th bit becomes 0
- All other bits remain unchanged

Exactly what we want.

⚠ Edge Cases

1. Bit Already Clear

```
n = 9
1001
```

Clear bit 2:

```
1001
1011
----
1001
```

Result remains:

```
9
```

2. Clearing the 0th Bit

```
n = 5
```

```
0101
1110
----
0100
```

Result:

```
4
```

3. Number is Zero

```
n = 0
```

Any bit cleared:

```
0
```

Result remains:

```
0
```

 Complexity

Operation	Complexity
Time	O(1)
Space	O(1)

 Interview Takeaway

Remember the four most important bit manipulation formulas:

Operation	Formula
Check Bit	$n \& (1 \ll k)$
Set Bit	$n \mid (1 \ll k)$
Clear Bit	$n \& \sim(1 \ll k)$
Toggle Bit	$n \wedge (1 \ll k)$

A very common beginner mistake is:

```
n | ~(1<<k)
```

because OR does **not clear** bits.

To clear a bit, always think:

```
Clear  
=  
AND  
+  
NOT Mask
```

Memory Trick

Imagine the mask as a stencil 🧐.

```
1 → Keep the bit  
0 → Erase the bit
```

After inversion:

```
11101111
```

the target bit position contains a hole (0).

When AND is applied:

```
n & ~(1<<k)
```

that bit gets erased while everything else survives.

Remember:

```
Clear Bit  
=  
AND with Inverted Mask
```

```
n & ~(1<<k)
```

Golden Memory Rule

```
Check  → AND
Set    → OR
Clear  → AND + NOT
Toggle → XOR
```

Master these four formulas, and nearly every beginner bit manipulation problem becomes straightforward.

5 Update ith Bit

 **Goal:** Update the *ith* bit of a number to a given value (*0* or *1*).

Problem

Given:

- A number *n*
- A bit position *i*
- A new bit value *newBit* (*0* or *1*)

Update the *ith* bit of *n* so that it becomes exactly *newBit*.

Why Is This Problem Interesting?

Setting a bit is easy.

Clearing a bit is easy.

But what if you don't know the current value of the bit?

The bit could already be:

- *0*
- *1*

We need a solution that works in **both cases**.

Golden Trick

Updating a bit is a **two-step process**:

```
Update Bit
=
Clear Bit
+
Set New Value
```

First erase the old bit.

Then insert the new bit.



Important Logic

Step 1: Clear ith Bit

```
n = n & ~(1 << i);
```

Step 2: Set New Value

```
n = n | (newBit << i);
```

Combined Formula

```
n = (n & ~(1 << i)) | (newBit << i);
```



Intuition

Imagine the bit position as a whiteboard.

Suppose the whiteboard currently contains:

```
1
```

or

```
0
```

You want to write a new value.

Would you directly write over it?

✗ No.

First erase the board.

Then write the new value.

```
Old Bit
  ↓
Erase
  ↓
Write New Bit
```

Bit updating works exactly the same way.

✂ Dry Run

Example

```
n = 13

Binary:
1101

i = 2
newBit = 0
```

Step 1: Clear Bit

Mask:

```
1 << 2

0100
```

Inverse Mask:

```
1011
```

Operation:

```
1101
1011
----
1001
```

Now bit 2 is definitely 0.

Step 2: Insert New Value

```
newBit = 0
```

Shift:

```
0000
```

OR Operation:

```
1001
0000
----
1001
```

Final Answer

```
1001 = 9
```

☒ Bit updated successfully.

✗ Common Mistake

Many beginners try:

```
n ^(newBit <<i>i
```

This is incorrect.

Why?

Because XOR **toggles** the bit.


```
1 → 0  
0 → 1
```

But updating means:

```
Set exactly to 0  
or  
Set exactly to 1
```

These are different operations.

💧 Why It Works

Clear Step

```
n & ~(1 << i)
```

Forces the *i*th bit to become:

```
0
```

regardless of its previous value.

Set Step

```
n | (newBit << i)
```

If:

```
newBit = 0
```

nothing changes.

If:

```
newBit = 1
```

the bit becomes 1.

Thus the final value of the bit is always exactly newBit.

⚠ Edge Cases

Case 1: Bit already equals newBit

```
Current Bit = 1
newBit = 1
```

Still works correctly.

Case 2: Updating to 0

```
Current Bit = 1
newBit = 0
```

Bit gets cleared.

Case 3: Updating to 1

```
Current Bit = 0
newBit = 1
```

Bit gets set.

Case 4: Updating Most Significant Bit

The same logic works.

No special handling is required.

🕒 Complexity

Operation	Complexity
Time	O(1)
Space	O(1)

Only a few bitwise operations are performed.

Interview Takeaway

Whenever you hear:

"Change a bit to a specific value"

think:

```
CLEAR
+
SET
```

The standard formula is:

```
(n & ~(1<<i)) | (newBit <<i)
```

This is the most common interview solution.

Memory Trick

```
UPDATE
↓
ERASE
↓
WRITE
```

Or simply:

```
Update Bit
=
Clear Bit
+
Set Bit
```

 First remove the old bit.

 Then write the new bit.

6 Toggle ith Bit

Problem: Toggle the K-th Bit

Given an integer n and a bit position k , toggle the K -th bit of the number and return the resulting value.

Toggling means:

```
0 → 1
1 → 0
```

⚡ Golden Trick

Use the XOR operator with a mask containing only the K -th bit set.

```
n ^ (1 << k)
```

XOR automatically flips the target bit while leaving all other bits unchanged.

🧠 Intuition

To toggle a bit, we need something that can:

```
Turn 0 into 1
Turn 1 into 0
```

The XOR operator has exactly this property:

```
0 ^ 1 = 1
1 ^ 1 = 0
```

So we create a mask having only the K -th bit set:

```
1 << k
```

and XOR it with the number.

🔮 Dry Run

Input

```
n = 13  
k = 2
```

Binary representation:

```
13 = 1101
```

Step 1: Create Mask

```
1 << 2 = 0100
```

Step 2: Apply XOR

```
1101  
0100  
----  
1001
```

Result

```
9
```

The 2nd bit changed from:

```
1 → 0
```

✖ Counter Example

Toggle the same bit twice.

First Toggle

```
13 = 1101
```

```
1101
0100
----
1001
```

Result:

```
9
```

Second Toggle

```
1001
0100
----
1101
```

Result:

```
13
```

Notice that toggling the same bit twice restores the original number.

Why It Works

XOR has a special property:

```
0 ^ 1 = 1
1 ^ 1 = 0
```

and

```
0 ^ 0 = 0
1 ^ 0 = 1
```

The mask contains:

```
0 everywhere
1 only at the K-th bit
```

Therefore:

- The K-th bit gets flipped.
- All other bits remain unchanged.

⚠ Common Mistake

Many beginners confuse XOR with OR.

Wrong

```
n | (1<<k)
```

This only sets the bit.

```
0 → 1
1 → 1
```

The bit never changes from 1 back to 0.

Correct

```
n ^ (1<<k)
```

This truly toggles the bit.

```
0 → 1
1 → 0
```

⚠ Edge Cases

1. Toggle a Set Bit

```
n = 13  
1101
```

Toggle bit 2:

```
1101  
0100  
----  
1001
```

Result:

```
9
```

2. Toggle an Unset Bit

```
n = 9  
1001
```

Toggle bit 2:

```
1001  
0100  
----  
1101
```

Result:

```
13
```

3. Toggle the 0th Bit

```
n = 10  
1010
```

Toggle bit 0:


```
1010
0001
----
1011
```

Result:

```
11
```

4. Number is Zero

```
n = 0
k = 3
```

```
0000
1000
----
1000
```

Result:

```
8
```

 Complexity

Operation	Complexity
Time	O(1)
Space	O(1)

 Interview Takeaway

The four most important bit manipulation formulas are:

Operation	Formula
-----------	---------

Operation	Formula
Check Bit	$n \& (1 \ll k)$
Set Bit	$n \mid (1 \ll k)$
Clear Bit	$n \& \sim(1 \ll k)$
Toggle Bit	$n \wedge (1 \ll k)$

Among these, XOR is the only operation that flips a bit.

Memory Trick

Think of XOR as a switch .

Every time you press it:

OFF → ON
ON → OFF

Press it once:

0 → 1
1 → 0

Press it again:

1 → 0
0 → 1

That's why:

Toggle Bit
=
XOR with Shifted 1

$n \wedge (1 \ll k)$

Golden Memory Rule

```
Check  → AND
Set    → OR
Clear  → AND + NOT
Toggle → XOR
```

If OR is a marker  that permanently writes a **1**, then XOR is a switch  that flips the bit every time you touch it.

Clear Last *i* Bits

Problem

Given a number **n** and an integer **i**, clear (set to 0) the last **i** bits of the number.

Golden Trick

Create a mask having:

```
1s on the left
0s on the last i positions
```

Then perform an AND operation.

```
n &((-1)<<i)
```

Important Logic

Create Mask

```
(-1)<<i
```

Clear Last *i* Bits

```
n &((-1)<<i)
```

Intuition

The AND operator preserves bits where the mask contains **1** and clears bits where the mask contains **0**.

By left-shifting **-1**, we get:

```
11111111111111000
```

for **i = 3**.

The last 3 bits become **0**, so they get cleared after the AND operation.

Dry Run

Input

```
n = 29
i = 3
```

Binary Representation

```
29 = 11101
```

Mask

```
(-1 << 3)
11111000
```

AND Operation

```
11101
11111000
-----
11000
```

Result

```
24
```

✖ Counter Example

Using only:

```
n << i
```

does **not** clear bits.

Example:

```
n = 15  
  
1111 << 2  
  
111100
```

This shifts the entire number and changes its value completely.

💧 Why It Works

The mask has:

```
1 → Keep the bit  
0 → Clear the bit
```

Since the last i positions contain 0 , AND forces those bits to become 0 .

```
x & 1 = x  
x & 0 = 0
```

⚠ Edge Cases

$i = 0$

```
(-1) << 0
```

No bits are cleared.

$n = 0$

Result remains:

```
0
```

$i \geq$ Number of Significant Bits

All meaningful bits may become 0.

Complexity

Operation	Complexity
Time	$O(1)$
Space	$O(1)$

Interview Takeaway

Whenever you need to clear multiple bits from the right side, create a mask with:

```
(-1)<<i
```

and perform an AND operation.

This avoids loops and works in constant time.

Memory Trick

Think of the mask as a giant wall of 1s with the last i positions punched out:

```
11111111000
  ↑
holes
```

When AND is applied, the holes erase the last i bits of the number.

Clear Bits in a Given Range

Problem

Given a number n and two indices i and j , clear (set to 0) all bits from position i to j (inclusive).

⚡ Golden Trick

Create a bitmask having:

```
1s before j  
0s from i to j  
1s after i
```

Then perform an AND operation.

```
n & bitmask
```

💻 Important Logic

Left Part of Mask

```
(~0)<<(j + 1)
```

Right Part of Mask

```
(1<<i)-1
```

Combine Both Parts

```
a | b
```

Clear the Range

```
n & bitmask
```

🧠 Intuition

To clear a range of bits, we need a mask that preserves everything except the target range.

For example, if:

```
i = 2  
j = 5
```

We want a mask like:

```
11000011
```

Notice:

```
Bits 5,4,3,2 → 0  
Remaining bits → 1
```

When AND is applied, only the bits in the specified range become 0.

✂ Dry Run

Input

```
n = 255  
i = 2  
j = 5
```

Binary Representation

```
255 = 11111111
```

Left Mask

```
(~0) << (5 + 1)  
  
11000000
```

Right Mask

```
(1 << 2) - 1  
  
00000011
```


Final Mask

```
11000000
00000011
-----
11000011
```

Apply AND

```
11111111
11000011
-----
11000011
```

Result

```
195
```

✘ Counter Example

Using only:

```
(~0)<<(j +1)
```

produces:

```
11000000
```

This also clears bits below `i`, which is incorrect.

We need the right-side mask to preserve those bits.

💧 Why It Works

The mask is divided into three regions:

```
111111 | 0000 | 111
```

- Left **1**s preserve higher bits.
- Middle **0**s clear the target range.
- Right **1**s preserve lower bits.

Since:

```
x & 1 = x
x & 0 = 0
```

only the bits from **i** to **j** get erased.

⚠ Edge Cases

i = j

Only one bit is cleared.

n = 0

Result remains:

```
0
```

i = 0

All bits from the least significant bit up to **j** are cleared.

j = 31 (for int)

Be careful with shift operations on 32-bit integers.

🕒 Complexity

Operation	Complexity
Time	$O(1)$
Space	$O(1)$

🎓 Interview Takeaway

Range-based bit clearing is usually solved by creating two masks:

```
Left Mask  → Preserve higher bits
Right Mask → Preserve lower bits
```

Combining them with OR creates a gap of zeros exactly where the bits need to be cleared.

🧩 Memory Trick

Imagine building a bridge around the bits you want to destroy:

```
111111  0000  111
Safe    Clear Safe
```

The left and right sides protect the bits you want to keep, while the middle gap wipes out the selected range.

9 Check if Number is Power of 2

🔗 Problem

Determine whether a given number is a Power of 2.

⚡ Golden Trick

```
(n & (n - 1)) == 0
```

💻 Java Function

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

🧐 Key Observation

A Power of 2 always contains **exactly one Set Bit**.

Examples

```
1  = 0001
2  = 0010
4  = 0100
8  = 1000
```

```
16 = 10000
32 = 100000
```

Notice that every number has only **one 1** in its binary representation.

🔍 What is a Set Bit?

A bit whose value is **1** is called a **Set Bit**.

Example

```
13 = 1101
```

```
1 1 0 1
↑ ↑   ↑
```

Number of Set Bits = **3**

💡 Intuition

When we subtract **1** from a number:

- The rightmost Set Bit becomes **0**
- All bits to its right become **1**

For a Power of 2, there is only one Set Bit.

So subtracting **1** completely removes that Set Bit.

🔮 Dry Run

Example: $n = 8$

Binary Representation:

```
8 = 1000
```

Subtract 1:

```
7 = 0111
```

Perform AND Operation:

```
1000
0111
----
0000
```

Result = 0 ☒

Therefore, **8 is a Power of 2.**

Another Example

Example: n = 16

```
16 = 10000
15 = 01111
```

```
10000
01111
-----
00000
```

Result = 0 ☒

Therefore, **16 is a Power of 2.**

Non-Power of 2 Example

Example: n = 10

```
10 = 1010
9 = 1001
```

```
1010
1001
----
1000
```

Result $\neq$ 0

Therefore, **10 is NOT a Power of 2**.

💧 Why Does This Work?

The expression:

```
n & (n - 1)
```

always removes the **rightmost Set Bit** from a number.

Example

```
12 = 1100  
11 = 1011
```

```
1100  
1011  
----  
1000
```

The rightmost Set Bit is removed.

For a Power of 2, there is only one Set Bit.

Removing it leaves:

```
0000
```

Hence:

```
(n & (n - 1)) == 0
```

⚠ Important Edge Case

What about 0?

```
0 & (-1) = 0
```

But:

```
0 is NOT a Power of 2
```

That's why we add:

```
n > 0
```

Final Condition:

```
n > 0 && (n & (n - 1)) == 0
```

Complexity Analysis

Time Complexity

```
O(1)
```

Only one bitwise operation is performed.

Space Complexity

```
O(1)
```

No extra memory is used.

Interview Takeaway

```
Power of 2 → Exactly One Set Bit
```

```
n & (n - 1) = 0
```

Quick Examples

```
1  ☒  
2  ☒  
4  ☒
```

8	☒
16	☒
32	☒
3	☒
5	☒
10	☒
12	☒
18	☒

Memory Trick

Imagine a Power of 2 as a room with only **one light turned on**.

1000

Subtracting 1 turns that light off and switches all lower lights on.

0111

Now there is no position where both numbers have a 1.

1000
0111

0000

Result = 0

Therefore, the number is a Power of 2. 

10 Count Set Bits

Problem

Given a non-negative integer **N**, count the number of **set bits (1s)** present in its binary representation.

A set bit is a bit whose value is 1.

Example


```
N = 13  
  
Binary = 1101  
  
Set Bits = 3
```

⚡ Golden Trick

```
n = n &(n - 1);
```

This operation removes the **rightmost set bit** from a number.

Each iteration removes exactly one **1**.

Therefore:

```
Number of iterations = Number of set bits
```

💻 Important Logic

Approach 1: Check Every Bit

```
for(int i = 0;  
i < 31; i++){  
    if((n &(1<<i)) != 0){  
        count++;  
    }  
}
```

Approach 2: Check Last Bit Repeatedly

```
while(n > 0){  
    count += (n & 1);  
    n >>= 1;  
}
```

Approach 3: Brian Kernighan's Algorithm ★

```
while(n > 0){  
    n = n & (n - 1);  
    count++;  
}
```

Intuition

Every set bit contributes **1** to the final count.

Instead of checking every position individually, we can repeatedly remove set bits until the number becomes zero.

Consider:

```
1101
```

If we remove one set bit at a time:

```
1101  
↓  
1100  
↓  
1000  
↓  
0000
```

We removed:

```
3 set bits
```

Hence the answer is:

```
3
```

Dry Run

Example: N = 13

```
Binary = 1101
```

Iteration 1

```
n      = 1101
n - 1  = 1100
```

```
1101
1100
----
1100
```

Count = 1

Iteration 2

```
n      = 1100
n - 1  = 1011
```

```
1100
1011
----
1000
```

Count = 2

Iteration 3

```
n      = 1000
n - 1  = 0111
```

```
1000
0111
----
0000
```

Count = 3

Number becomes:

```
0000
```

Stop.

☒ Answer = 3

✗ Why Not Check Every Bit?

A common solution is:

```
for(int i = 0;
i < 31; i++)
```

This works perfectly.

However, it always performs:

```
31 iterations
```

even when only a few bits are set.

Example:

```
10000000000000000000000000000000
```

contains only:

```
1 set bit
```

Yet the loop still checks all 31 positions.

💧 Why Brian Kernighan's Algorithm Is Better

Observe:

```
n & (n - 1)
```

always removes exactly one set bit.

Example:

```
1101 → 1100
1100 → 1000
1000 → 0000
```

One set bit disappears in every iteration.

Therefore:

```
Iterations = Number of Set Bits
```

Not:

```
Iterations = Total Number of Bits
```

This makes it extremely efficient when the number contains only a few set bits.

⚠ Edge Cases

1. Number is Zero

```
N = 0
```

```
Binary = 0
```

```
Set Bits = 0
```

2. Number is One

```
N = 1
```

```
Binary = 1
```

```
Set Bits = 1
```

3. All Bits Set

```
N = 15
```

```
Binary = 1111
```

```
Set Bits = 4
```

4. Large Power of Two

```
N = 1024
```

```
Binary = 10000000000
```

```
Set Bits = 1
```

5. Maximum Integer Value

```
N = 2147483647
```

```
Binary = 31 ones
```

```
Set Bits = 31
```

Complexity

Approach	Time Complexity	Space Complexity
Check Every Bit	$O(31)$	$O(1)$
Shift and Check	$O(\text{Number of Bits})$	$O(1)$
Brian Kernighan	$O(\text{Number of Set Bits})$	$O(1)$

Interview Takeaway

Whenever you hear:

```
Count Set Bits
```

Think:

```
n & (n - 1)
```

because it removes the rightmost set bit in one operation.

This is one of the most frequently asked bit manipulation tricks in coding interviews and competitive programming.

🧩 Memory Trick

Imagine every set bit is a glowing bulb 💡 :

```
11010100
```

Normal Approach

🔍 Check every position

You inspect every bulb, whether it's ON or OFF.

Brian Kernighan

🔧 Remove one glowing bulb at a time

```
11010100
↓
11010000
↓
11000000
↓
10000000
↓
00000000
```

Each removal increases the count by 1.

Golden Formula

```
n = n &(n - 1);
```

💡 **One operation = One set bit removed**

11 XOR Properties

Important Rules

```
a ^ a = 0  
  
a ^ 0 = a  
  
0 ^ a = a  
  
XOR is commutative
```

1 2 Swap Two Numbers Using XOR

⚡ Logic

```
a = a ^ b;  
b = a ^ b;  
a = a ^ b;
```

💡 Intuition

XOR has a magical cancelation property:

```
x ^ x = 0  
x ^ 0 = x
```

Using this, we temporarily hide both values inside XOR operations and then recover them in reverse order without using any extra variable.

Think of XOR like a reversible binary locker 🔒

🔄 How It Works

Initial Values

```
a = 5  
b = 3
```

Step 1

```
a = a ^ b;
```


Now:

```
a = 6  
b = 3
```

a stores the XOR combination of both numbers.

Step 2

```
b = a ^ b;
```

Now:

```
a = 6  
b = 5
```

Original value of **a** is restored into **b**.

Step 3

```
a = a ^ b;
```

Now:

```
a = 3  
b = 5
```

Original value of **b** is restored into **a**.

☒ Numbers swapped successfully.

✗ Mistake I Made Initially

Wrong Logic

```
a = a ^ a ^ b;  
b = b ^ b ^ a;
```

⊘ Why It Failed

Since:

```
x ^ x = 0
```

The first statement becomes:

```
a = 0^b;  
a = b;
```

The original value of `a` gets destroyed immediately.
Then the second statement also fails because both variables start becoming the same value.

Example

```
Initial:  
a = 5  
b = 3
```

After wrong logic:

```
a = 3  
b = 3
```

✗ Swapping does not happen.

🕒 Complexity

Complexity	Value
Time Complexity	$O(1)$
Space Complexity	$O(1)$

📌 Important Note

Although XOR swap is a famous bit manipulation trick, real-world code usually prefers using a temporary variable because it is easier to read and debug.

```
int temp = a;  
a = b;  
b = temp;
```

13 Find Unique Element Using XOR

Problem

Every element appears twice except one.

Trick

Use XOR on all elements.

Intuition

Duplicate elements cancel each other.

Logic

$$x \oplus x = 0$$

Only unique element remains.

14 Set the Rightmost Unset Bit

Problem: Set the Rightmost Unset Bit

Given a non-negative integer **N**, set its **rightmost unset (0) bit** and return the resulting number.

Examples

Example 1

Input: N = 10

Binary:

```
1010

Rightmost unset bit:
1010
  ↑

After setting:
1011

Output: 11
```

Example 2

```
Input:  N = 7

Binary:
111

After setting the rightmost unset bit:
1111

Output: 15
```

⚡ Golden Trick

```
n | (n + 1)
```

This single expression sets the **rightmost unset bit** of a number.

🧠 Intuition

To understand the trick, let's first see what happens when we add **1**.

Consider:

```
N = 10

Binary:
1010
```

Adding **1**:

```
1010
+   1
----
1011
```

Notice something interesting:

```
1010
1011
  ↑
```

The rightmost unset bit became 1.

Now perform OR:

```
1010
1011
----
1011
```

The OR operation preserves all existing 1s and ensures that the rightmost unset bit becomes 1.

Dry Run

Example: $N = 10$

Step 1

```
N = 10

Binary:
1010
```

Step 2

```
N + 1

1010
+   1
----
1011
```

Step 3

Apply OR:

```
1010
1011
----
1011
```

Step 4

Convert back to decimal:

```
1011 = 11
```

☒ Answer = 11

✗ Why Can't We Just Use $n + 1$?

Many beginners think:

```
return n + 1;
```

should work.

It works for some cases, but completely fails for others.

Case 1: Works Accidentally

```
N = 10
```

```
1010
```

```
N + 1 = 1011
```

Output:

```
11
```

Looks correct.

Case 2: Fails

```
N = 11
```

```
Binary:  
1011
```

Adding 1:

```
1011  
+  1  
----  
1100
```

Result:

```
12
```

But the expected answer is:

```
1011  
  
Rightmost unset bit:  
1011  
  ↑  
  
Set it:  
  
1111 = 15
```

Comparison

```
Using n + 1      → 1100 (12) ✗  
Using n|(n+1)    → 1111 (15) ✓
```

Why?

Because $n + 1$ may change multiple bits due to carry propagation.

```
1011 + 1
```

```
1 + 1 = 0 (carry)
```

```
1 + carry = 0 (carry)
```

```
0 + carry = 1
```

Result:

```
1100
```

Several bits changed.

But our goal is only:

```
Set the first 0 bit from the right.
```

The OR operation guarantees exactly that.

Why Does $n \mid (n + 1)$ Always Work?

Suppose:

```
N = xxxx01111
```

The rightmost unset bit is:

```
xxxx0 1111
```

```
↑
```

Adding 1:

```
xxxx10000
```

Observe:

1. The rightmost unset bit becomes 1.
2. All bits to its right become 0.

Now OR them:

```
xxxx01111
xxxx10000
-----
xxxx11111
```

Result:

```
The rightmost unset bit is set.
All existing 1s remain unchanged.
```

Exactly what we need.

Why It Works

When we compute:

```
n | (n + 1)
```

- $n + 1$ turns the first 0 bit (from the right) into 1.
- OR keeps every existing 1.
- OR also keeps the newly created 1.

Therefore:

```
The rightmost unset bit becomes set.
```

This works in a single operation.

Edge Cases

1. Number is 0

```
0 -> 1
```

2. Single Bit Number

1 -> 3

1
↓
11

3. Multiple Trailing Ones

11

1011

Result:

1111 = 15

4. All Bits Are Set

7

111

Result:

1111 = 15

5. Large Power of Two

1024

1000000000

Result:

```
10000000001
```

6. Integer Overflow Case

```
Integer.MAX_VALUE
```

```
2147483647

01111111111111111111111111111111
```

The next bit lies outside the positive `int` range.

Be careful if the platform includes this case.

 Complexity

Operation	Complexity
Time	O(1)
Space	O(1)

 Interview Takeaway

Whenever you hear:

```
Set the rightmost unset bit
```

Immediately think:

```
n | (n + 1)
```

Key observation:

```
n + 1
```

finds the first zero from the right and turns it into one.

Then:

```
OR
```

preserves all existing set bits while keeping that newly set bit.

This produces the desired answer in constant time.

Memory Trick

Imagine a binary number as a row of switches:

```
1011
```

You want to turn ON the first OFF switch from the right.

```
1011
  ↑
```

$n + 1$ locates that switch and flips it ON.

```
1100
```

Then OR merges the old and new states:

```
1011
1100
----
1111
```

Memory Formula

```
Set Rightmost
Unset Bit
```

```
=  
n | (n + 1)
```

Whenever you see:

Rightmost 0 → Make it 1

Think:

ADD 1 → OR



14 Remove Last Set Bit

⚡ Trick

```
n & (n - 1)
```

💡 Intuition

Subtracting 1 flips bits after the last set bit.

15 Check kth Bit Set or Not



Problem: Check Whether the K-th Bit is Set

Given an integer `n` and a bit position `k`, determine whether the K-th bit of the number is set (1) or not set (0).

⚡ Golden Trick

Create a mask having only the K-th bit set.

```
1 << k
```

Then perform a bitwise AND operation:

```
n & (1 << k)
```

- Result = 0 → K-th bit is NOT set
- Result ≠ 0 → K-th bit IS set

Intuition

Imagine each bit of a number is a switch.

For example:

```
13 = 1101
```

```
Position: 3 2 1 0  
Bits:      1 1 0 1
```

To check a specific switch, we create a mask that turns ON only that position.

For K = 2:

```
0001 << 2 = 0100
```

Now perform AND:

```
1101  
0100  
----  
0100
```

Since the result is not zero, the bit exists and is set.

Dry Run

Input

```
n = 13  
k = 2
```

Step 1

Create mask:

```
1 << 2 = 0100
```

Step 2

Perform AND:

```
1101  
0100  
----  
0100
```

Step 3

Result $\neq$ 0

```
Answer = Set
```

✖ Counter Example

Input

```
n = 13  
k = 1
```

Binary:

```
13 = 1101
```

Mask:

```
0010
```

AND:

```
1101
0010
----
0000
```

Result = 0

Answer = Not Set

Why It Works

The mask contains exactly one bit set:

```
0001 << k
```

When AND is performed:

```
n & mask
```

all bits become zero except the K-th bit.

Therefore:

- If K-th bit was 1 → result is non-zero.
- If K-th bit was 0 → result is zero.

This directly tells us whether the bit is set.

Edge Cases

1. Checking 0th Bit


```
n = 5 (0101)
k = 0
```

Result:

```
Set
```

2. Number is Zero

```
n = 0
```

Every bit is unset.

Result:

```
Not Set
```

3. Large Bit Position

```
k >= 31
```

For Java `int`, valid positions are typically:

```
0 to 31
```

 Complexity

Operation	Complexity
Time	O(1)
Space	O(1)

 Interview Takeaway

Whenever you need to inspect a specific bit:

```
(n & (1 << k)) != 0
```

This is the most common and efficient technique used in bit manipulation problems.

Memory Trick

Think of the mask as a spotlight .

```
1 << k
```

moves the spotlight to the K-th position.

```
n & mask
```

illuminates only that bit.

- Light visible → Bit is Set ☒
- No light → Bit is Not Set ✕

Remember:

```
Check Bit
    =
    AND
with Shifted 1
```

```
(n & (1 << k))
```

16 Fast Exponentiation (Binary Exponentiation)

Problem

Given a number a and a power n , calculate:

```
a^n
```

efficiently without multiplying a exactly n times.

⚡ Golden Trick

Every number can be represented in binary.

Instead of computing:

$$a \times a \times a \times a \times \dots$$

use the binary representation of the exponent and repeatedly square the base.

💻 Approach 1: My Method

Logic Snippet

```
if(power %2==0){
    number *=number;
    power >>=1;
}
else{
    mul *=number;
    power--;
}
```

Idea

- If the power is even, square the number and halve the power.
 - If the power is odd, multiply the answer by the current number and make the power even by subtracting 1.
 - Continue until power becomes 0.
-

💻 Approach 2: Standard Binary Exponentiation

Logic Snippet

```
if((power &1)!=0){
    ans *=number;
}

number *=number;
power >>=1;
```

Idea

- Check the last bit of the exponent.
- If it is 1, include the current power in the answer.
- Square the base to generate the next power.
- Right shift the exponent to process the next binary digit.

Intuition

Suppose:

$$3^{13}$$

Binary representation of 13:

$$1101_2$$

which means:

$$13 = 8 + 4 + 1$$

Therefore:

$$3^{13} = 3^8 \times 3^4 \times 3^1$$

The algorithm only multiplies the powers corresponding to set bits.

Dry Run

Input

```
number = 3  
power = 13
```

Binary Form

$$13 = 1101_2$$

Powers Generated

$$\begin{aligned}3^1 &= 3 \\3^2 &= 9 \\3^4 &= 81 \\3^8 &= 6561\end{aligned}$$

Required Powers

$$\begin{array}{c}1101_2 \\ \uparrow \uparrow \uparrow \\ 3^8 \times 3^4 \times 3^1\end{array}$$

Result

$$\begin{aligned}3 \times 81 \times 6561 \\ = \\ 1594323\end{aligned}$$

✗ Brute Force Approach

Logic

Multiply the number exactly n times.

Example:

$$\begin{aligned}2^{10} \\ = \\ 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2\end{aligned}$$

This requires many unnecessary multiplications.

💧 Why It Works

Every exponent can be decomposed into powers of 2.

Example:

$$13 = 1101_2$$

$$13 = 8 + 4 + 1$$

Thus:

$$a^{13} = a^8 \times a^4 \times a^1$$

The algorithm:

1. Generates powers by squaring.
2. Uses binary bits to decide which powers are needed.
3. Multiplies only those powers.

Edge Cases

Power = 0

$$a^0 = 1$$

Base = 0

$$0^n = 0$$

for $n > 0$

Base = 1

$$1^n = 1$$

Power = 1

$$a^1 = a$$

Comparison

Approach	Time Complexity
Brute Force	$O(n)$
Your Approach	$O(\log n)$
Standard Binary Exponentiation	$O(\log n)$

Complexity

Operation	Complexity
Time	$O(\log n)$
Space	$O(1)$

Interview Takeaway

Fast Exponentiation is one of the most important applications of bit manipulation.

The key observation is:

Exponent $\rightarrow$ Binary Representation

and

Repeated Squaring $\rightarrow$ Generate Powers of 2

Combining both reduces the complexity from:

$O(n)$

to

$O(\log n)$

Memory Trick

Think of the exponent as a treasure map written in binary.

$13 = 1101_2$

Each 1 says:

```
"Pick this power."
```

Each step:

```
Square the base
↓
Move to next bit
↓
Pick power if bit = 1
```

This allows us to reach huge powers using only a handful of operations.

17 Minimum Bit Flips to Convert Number

Problem

Given two integers `start` and `goal`, determine the **minimum number of bit flips** required to convert `start` into `goal`.

Example

```
start = 10
goal  = 7

10 = 1010
7  = 0111

Answer = 3
```

Golden Trick

XOR reveals differences

```
start ^goal
```

- Same bits → 0
- Different bits → 1

The resulting number contains **1s** exactly at the positions that need to be flipped.

Count Set Bits Efficiently

```
n = n &(n - 1);
```

This removes the **rightmost set bit** in every iteration.

The number of iterations equals the number of set bits.



Code

Find Different Bit Positions

```
int xor = start ^ goal;
```

Count Set Bits

```
while(xor > 0){  
    xor = xor &(xor - 1);  
    count++;  
}
```



Intuition

Think of XOR as a **difference detector**.

```
1010  
0111  
----  
1101
```

Every **1** in the XOR result represents a bit position where the two numbers differ.

Now the task becomes:

Count how many **1s** exist in the XOR result.

Instead of checking all 32 bits one by one, repeatedly remove the rightmost set bit until the number becomes zero.

🔮 Dry Run

Input

```
start = 10  
goal  = 7
```

Step 1: XOR

```
1010  
0111  
----  
1101
```

```
xor = 13
```

Iteration 1

```
1101 & 1100 = 1100
```

```
count = 1
```

Iteration 2

```
1100 & 1011 = 1000
```

```
count = 2
```

Iteration 3

```
1000 & 0111 = 0000
```

```
count = 3
```

Loop ends.

Output

```
3
```

✗ Counter Example

Wrong Approach

Checking every bit individually:

```
for(int i = 0;
i < 32; i++)
```

This works, but it examines all bits even when only a few bits differ.

Example:

```
10000000000000000000000000000000
```

Only one bit is set, yet all 32 positions are checked.

💡 Why It Works

XOR has a special property:

```
0 ^ 0 = 0
1 ^ 1 = 0
0 ^ 1 = 1
1 ^ 0 = 1
```

Therefore:

```
start ^ goal
```

creates a number whose set bits represent exactly the positions that must be flipped.

Brian Kernighan's Algorithm:

```
n &(n - 1)
```

removes one set bit at a time.

Thus:

```
Number of iterations  
=  
Number of required bit flips
```

⚠ Edge Cases

Same Numbers

```
start = 5  
goal  = 5
```

```
XOR = 0  
Answer = 0
```

One Number is Zero

```
start = 0  
goal  = 7
```

```
0 = 0000  
7 = 0111
```

```
Answer = 3
```

Single Bit Difference

```
start = 8  
goal  = 0
```

```
1000 → 0000
```

```
Answer = 1
```

Complexity

Time Complexity

```
O(k)
```

where k = number of set bits in $(start \oplus goal)$.

Space Complexity

```
O(1)
```

Interview Takeaway

Whenever an interviewer asks:

```
"How many bits differ between two numbers?"
```

Immediately think:

```
start ^goal
```

Whenever you need to count set bits efficiently:

```
n &(n -1)
```

Combining these two ideas gives the optimal solution.

🧩 Memory Trick

🔗 XOR Finds Differences

```
Different → 1
Same      → 0
```

🔗 AND with (n - 1) Removes One Set Bit

```
n & (n - 1)
```

💡 Remember:

```
Difference Detector
      +
Set Bit Counter
      =
Minimum Bit Flips
```

🔑 XOR Mastery

XOR is one of the most important tools in Bit Manipulation.

Why XOR is Powerful

- Cancels duplicates
- Helps in toggling
- Enables optimized solutions
- Useful in encryption
- Excellent for state manipulation

18 Add 1 to an Integer Using Bit Manipulation

🔒 Problem

Add 1 to a given integer without using the + operator.

⚡ Golden Trick

Use the relationship between **Bitwise NOT (~)** and **Two's Complement Representation**.

Logic Snippet

$$-(\sim n)$$

Intuition

Computers store signed integers using **Two's Complement**.

One important identity is:

$$\sim n = -(n + 1)$$

Rearranging:

$$n + 1 = -(\sim n)$$

Therefore, to add **1** to a number, we can simply:

$$-(\sim n)$$

without using the **+** operator.

Dry Run

Example 1

Input

$$n = 5$$

Binary Representation

$$5 = 00000101$$

Apply NOT

~ 5

11111010

This represents:

-6

Negate

 $-(-6)$

=

6

Result

 $5 + 1 = 6$

Example 2

Input

 $n = 10$

Apply Formula

 $-(\sim 10)$

=

11

Result

 $10 + 1 = 11$

✖ Brute Force Approach

Use the arithmetic operator:

```
n + 1
```

While simple, the goal of this problem is to understand how addition can be achieved using bit-level properties.

💡 Why It Works

We know:

```
~n = -(n + 1)
```

For example:

```
n = 7  
  
~7 = -8
```

Taking negative on both sides:

```
-(~7)  
=  
-(-8)  
=  
8
```

which is exactly:

```
7 + 1
```

Thus:

```
-(~n)
```

always produces:

```
n + 1
```

⚠ Edge Cases

$n = 0$

```
-(~0)
=
1
```

$n = -1$

```
-(~-1)
=
0
```

$n = \text{Integer.MAX_VALUE}$

Adding 1 causes integer overflow and wraps around according to Java's integer rules.

🕒 Complexity

Operation	Complexity
Time	$O(1)$
Space	$O(1)$

🎓 Interview Takeaway

This problem is less about practical coding and more about understanding:

- Two's Complement Representation
- Bitwise NOT Operator
- Relationships between arithmetic and bitwise operations

The key identity to remember is:

```
~n = -(n + 1)
```

which leads directly to:

$$n + 1 = -(\sim n)$$

Memory Trick

Remember the hidden formula:

$$\sim n \rightarrow -(n + 1)$$

So whenever you see:

$$-(\sim n)$$

think:

Minus of Minus Next Number

which simplifies to:

$$n + 1$$

- ☒ Flip all bits using $\sim$
- ☒ Negate the result using $-$
- ☒ You get the next integer

Pattern Recognition Guide

Observation	Think About
unique element	XOR
powers of 2	$n \ \& \ (n-1)$
subsets	bitmasking
state toggling	XOR
fast multiply/divide	shifts
counting bits	Kernighan Algorithm
masking required	AND / OR

✖ Common Mistakes

1. Forgetting Parentheses

Wrong:

```
n &1<<i
```

Correct:

```
n &(1<<i)
```

2. Confusing 0-Based Indexing

Bits are usually counted from right to left starting from 0.

3. Signed Right Shift Confusion

Negative numbers behave differently.

4. Overflow During Left Shift

Large shifts may overflow integer range.

5. Using XOR Without Understanding Cancellation

Always verify XOR behavior carefully.

19 Convert Uppercase Characters to Lowercase Using Bit Manipulation

Problem

Convert uppercase English letters (**A-Z**) to lowercase (**a-z**) using Bit Manipulation instead of built-in string or character conversion methods.

Golden Trick

The ASCII values of uppercase and lowercase letters differ by only **one bit**.

Logic Snippet

```
ch | ' '
```

Intuition

Let's compare the ASCII values of 'A' and 'a':

```
'A' = 65 = 01000001
'a' = 97 = 01100001
```

Notice that only the **5th bit** (counting from right, zero-based) is different:

```
01000001
01100001
  ↑
```

That bit is set in every lowercase letter.

When we OR an uppercase letter with a space character (' '):

```
' ' = 32 = 00100000
```

the required bit automatically becomes **1**, converting the character to lowercase.

Important Logic

Convert to Lowercase

```
ch | ' '
```

Equivalent Form

```
ch | 32
```

Dry Run

Example

Input

```
'A'
```

Binary Representation

```
'A' = 01000001
' ' = 00100000
```

OR Operation

```
01000001
00100000
-----
01100001
```

Result

```
'a'
```

Another Example

Input

```
'D'
```

Binary

```
'D' = 01000100
' ' = 00100000
```

OR Operation

```
01000100
00100000
-----
01100100
```

Result

```
'd'
```

✗ Brute Force Approach

Use built-in methods such as:

```
Character.toLowerCase(ch)
```

or

```
String.toLowerCase()
```

These are easier to use but hide the bit-level trick.

💡 Why It Works

ASCII stores uppercase and lowercase letters in a very clever way.

Uppercase

```
A = 01000001
B = 01000010
C = 01000011
```

Lowercase

```
a = 01100001
b = 01100010
c = 01100011
```

Only one bit changes:

```
00100000
```

OR operation guarantees that this bit becomes 1.

Since:

```
0 | 1 = 1
1 | 1 = 1
```

the character is converted to lowercase.

⚠ Edge Cases

Character Already Lowercase

```
'a' | ' '
=
'a'
```

No change occurs.

Digits

```
'5' | ' '
```

Produces an unexpected result.

This trick should only be used for English alphabetic characters.

Special Symbols

The trick is designed specifically for ASCII letters and may not work as expected for symbols.

🕒 Complexity

Operation	Complexity
Time	O(1) per character
Space	O(1)

For all 26 letters:

Operation	Complexity
Time	$O(26) \approx O(1)$
Space	$O(1)$

Interview Takeaway

ASCII uppercase and lowercase letters differ by exactly:

```
32
```

or

```
00100000
```

Therefore:

```
ch | ' '
```

sets the lowercase bit and converts an uppercase letter into its lowercase counterpart without using built-in functions.

Memory Trick

Think of the space character as a **Lowercase Switch**:

```
' ' = 00100000
```

Whenever you OR a letter with this switch:

```
A → a  
B → b  
C → c
```

because the lowercase bit gets turned ON.

```
ch | ' '
```

- ☒ Turn ON lowercase bit
- ☒ Preserve all other bits
- ☒ Instantly convert uppercase to lowercase

Complexity Cheatsheet

Operation	Time Complexity
Get Bit	$O(1)$
Set Bit	$O(1)$
Clear Bit	$O(1)$
Toggle Bit	$O(1)$
Power of 2 Check	$O(1)$
Count Set Bits	$O(\log n)$
Fast Exponentiation	$O(\log n)$

Real World Applications

Bit Manipulation is used in:

- Cryptography
 - Networking
 - Image Processing
 - Compression Algorithms
 - Operating Systems
 - Databases
 - Embedded Systems
 - Competitive Programming
 - Game Engines
 - Permission Systems
-

Rapid Revision Section

```
n & 1           -> Odd / Even
1 << i         -> Create mask
n | (1 << i)    -> Set ith bit
n & ~(1 << i)   -> Clear ith bit
n ^ (1 << i)    -> Toggle ith bit
n & (n - 1)    -> Remove last set bit
n & (n - 1) == 0 -> Power of 2
```

```
x << 1      -> Multiply by 2  
x >> 1      -> Divide by 2
```

Final Notes

Bit Manipulation is less about memorizing tricks and more about understanding how binary behaves.

Once the intuition becomes clear:

- many problems become shorter,
- optimized solutions become natural,
- and interview pattern recognition improves significantly.

Mastering Bit Manipulation builds strong low-level problem-solving intuition and improves coding efficiency across multiple domains.

Recommended Practice Order

1. Odd / Even
2. Get ith Bit
3. Set ith Bit
4. Clear ith Bit
5. Toggle ith Bit
6. Update ith Bit
7. Clear Last i Bits
8. Clear Range of Bits
9. Power of 2
10. Count Set Bits
11. XOR Problems
12. Unique Element
13. Fast Exponentiation

Language Used

- Java

Author

DSA practice repository focused on:

- Interview Preparation

- Placement Readiness
- Problem Solving
- Pattern Recognition
- Clean Documentation